



MASTERS IN MECHANICAL ENGINEERING

ROBOTIC MANIPULATION

Laboratory Project: Mako Surgical Robot

2nd Part: Dynamics and Control

Authors:

Pedro Vasques – ist1103665

Abdul Chowdhury – ist1112476

June 18, 2025

Contents

Introduction	3
1. Link Modeling and Physical Parameter Estimation	4
1.1. Assumptions and Structural Insights	4
1.2. Dynamic Modeling Theory and Link Properties	5
2. Dynamics Modeling: Newton-Euler Method	6
3. Worst Case Inertia Configurations & Decentralized PID Controllers	9
3.1. Joint Space Inertia Matrix Computation.....	9
3.2. Worst Case Scenario Inertia Configuration	9
3.3. Design and Implementation of Decentralized PID Joint Controllers.....	10
4. Centralized Inverse Dynamics Controller	13
4.1. Robot Dynamics Extraction.....	13
4.2. Control Law Description and Implementation	14
4.3. Direct Dynamics	14
4.4. Simulink Implementation	14
4.5. Controller Tuning and Validation.....	16
5. End Effector Task-Space Trajectory Desing and Controller Performance Comparison	17
5.1. End Effector Task-Space Trajectory Design	17
5.2. Controller Performance Comparison	19
Conclusion.....	21
References	22

Introduction

This report presents the dynamic modeling and control design of the Mako surgical robotic manipulator, developed as part of the coursework in Robotic Manipulation. The primary objective is to simulate and control the robot's motion using a systematic approach that integrates dynamic modeling, Simulink-based simulation, and classical control techniques.

The analysis begins with the estimation of each link's mass, center of mass, and inertia tensor. These properties are approximated using basic geometric shapes such as spheres, cylinders, and cubes, and serve as the foundation for dynamic calculations.

Following this, a complete dynamic model of the robot is developed using the Newton-Euler formulation. The symbolic torque expressions are derived in MATLAB and compiled into a numerical function. This function is implemented as a Simulink block that calculates the joint torques based on real-time joint states (positions, velocities, and accelerations).

To control the robot, both decentralized PID joint controllers and a centralized inverse dynamics controller are designed and tested. These controllers are integrated within the Simulink environment and evaluated based on their ability to track predefined motion trajectories under nominal and worst-case inertial conditions.

Finally, a task-space trajectory is assigned to the end-effector. The performance of each controller is compared in terms of accuracy, responsiveness, and robustness, using torque and trajectory-tracking data.

All models and simulations are implemented using original MATLAB and Simulink tools, ensuring a modular and comprehensive framework for analyzing the dynamic behavior and control of the Mako surgical robot.

1. Link Modeling and Physical Parameter Estimation

1.1. Assumptions and Structural Insights

To simplify modeling, each link of the Mako Surgical Robot is approximated using basic solid shapes like, Spheres, cylinders, and cubes—based on its geometry and function. All properties are centered at the link's center of mass, and inertia tensors are calculated accordingly.

From visual inspection and class notes, the following assumptions were made (Figure 1):

- β_1 (Body 1): Base body, modeled as sphere (not fixed)
- β_2 (Body 2): Body modeled as a sphere
- β_3 (Body 3): Body modeled along the y_0 -axis as cylinder
- β_4 (Body 4): Wrist joint 4 rotation link Body; modeled as a solid cube.
- β_5 (Body 5): Compact wrist component; modeled as cylinder.
- β_6 (Body 6): Final wrist link; modeled as a cylinder.

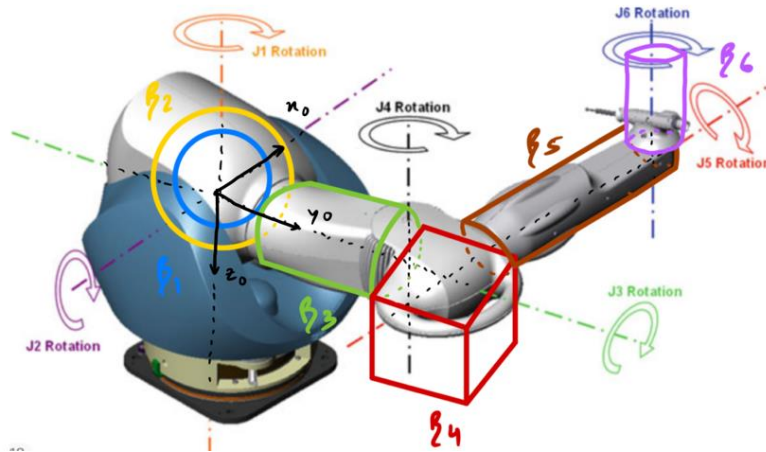


Figure 1: Shape and structure assumptions for Mako robot links

The joint axis reference frames, defined by DH parameters, are shown in Figure 2 to clarify the modeling assumptions.

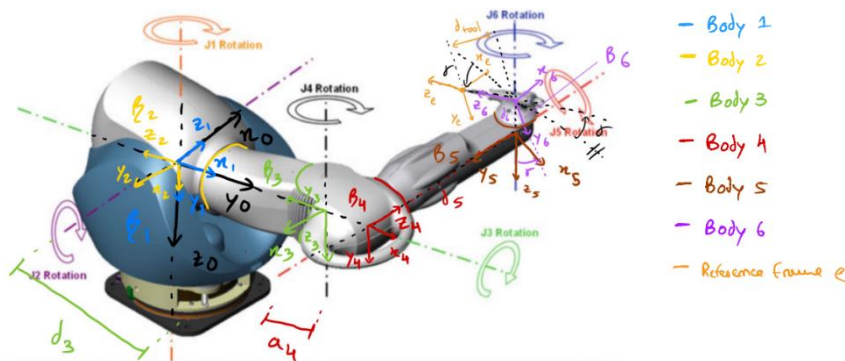


Figure 2: Mako Surgical Robot joint axis reference frames and DH parameters

1.2. Dynamic Modeling Theory and Link Properties

To simulate the robot's dynamics using the Newton-Euler formulation, each link must be characterized by three key properties:

- Mass (m_i): Total mass of the link.
- Center of Mass (C_{m_i}): The position of the center of mass (CoM) relative to the link's local frame.
- Inertia Tensor (I_i): A 3×3 matrix describing the mass distribution about the CoM.

To compute the inertia tensors, we approximate each link using simple solid shapes and apply standard analytical formulas:

- Solid Sphere: $I_x = I_y = I_z = (2/5)mr^2$ (1)

- Solid Cylinder: $I_x = I_y = \left(\frac{1}{12}\right)m(3r^2 + L^2), \quad I_z = (1/2)mr^2$ (2)

- Solid Cube: $I_x = I_y = I_z = (1/6)ma^2$ (3)

where:

m – mass of the solid

r – radius of the solid (sphere or cylinder)

L – length of the cylinder

a – cube side length

Using these formulas, the physical properties of the robot links were computed in MATLAB and summarized in Table 1.

Table 1: Body Shapes, Masses, Centers of Mass, and Inertia Tensors

Body	Shape	Mass [kg]	Center of Mass [m]	Inertia Tensor [kg·m ²]
1	Sphere	m_1	(0, 0, 0)	$\text{diag}(I_{x1}, I_{y1}, I_{z1})$
2	Sphere	m_2	(0, 0, 0)	$\text{diag}(I_{x2}, I_{y2}, I_{z2})$
3	Cilinder	m_3	(0, C_{y3} , 0)	$\text{diag}(I_{x3}, I_{y3}, I_{z3})$
4	Cube	m_4	(0, 0, 0)	$\text{diag}(I_{x4}, I_{y4}, I_{z4})$
5	Cilinder	m_5	(0, C_{y5} , 0)	$\text{diag}(I_{x5}, I_{y5}, I_{z5})$
6	Cilinder	m_6	(0, 0, $-C_{z6}$)	$\text{diag}(I_{x6}, I_{y6}, I_{z6})$

It is important to note that for links 3 and 6, a rotation adjustment is necessary because the axes of the defined center of mass are not aligned with the axes defined by the David-Hartenberg (DH) parameters. Without this correction, the dynamic model would produce inaccurate torque and force predictions due to frame inconsistencies. This equates to:

$$I_3 = R_{cg}^3 I_{cg} R_{cg}^3$$

$$I_6 = R_{cg}^6 I_{cg} R_{cg}^6$$

where:

$$R_{cg}^3 = R_{cg}^6 = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

The remaining inertia tensors are derived following the standard formulas corresponding for each body shape (see Equations (1)-(3)).

Throughout the report, to ensure realistic and consistent modeling, the constant dynamic properties (mass, center of mass and solid length parameters) of the Mako Surgical Robot (Table 3) were assumed to be as follows:

$$\begin{array}{llll} m_1 = 6.0 \text{ kg} & r_1 = 0.150 \text{ m} & & \\ m_2 = 4.0 \text{ kg} & r_2 = 0.100 \text{ m} & & \\ m_3 = 3.5 \text{ kg} & r_3 = 0.050 \text{ m} & L_3 = 0.300 \text{ m} (= d_3) & C_{y3} = 0.150 \text{ m} \\ \\ m_4 = 3.0 \text{ kg} & a_4 = 0.100 \text{ m} & & \\ m_5 = 3.0 \text{ kg} & r_5 = 0.030 \text{ m} & L_5 = 0.500 \text{ m} (= d_5) & C_{y5} = 0.250 \text{ m} \left(= \frac{L_5}{2} \right) \\ m_6 = 1.0 \text{ kg} & r_6 = 0.015 \text{ m} & L_6 = 0.115 \text{ m} (= d_{6p}) & C_{z6} = 0.0575 \text{ m} \left(= \frac{L_6}{2} \right) \\ \\ m_T = 20.5 \text{ kg} & & & \end{array}$$

It is important to note that realistic material properties such as density were not considered when assigning these values; the objective is to approximate dynamic behavior while maintaining computational simplicity and satisfying physical constraints such as total mass and size limits.

Given all the information above, the robot is modeled as having a static but non-fixed base, meaning the base does not translate (in the mako case) but can rotate in place. In the case of the Mako robot, this allows for rotational motion at the base joint while the overall base position remains stationary, affecting how base inertial properties contribute to the system's dynamics.

All calculated values were stored in the MATLAB file `mako_robot_mass_properties.mat` and will be used in the subsequent Simulink model for dynamic simulation and control design.

2. Dynamics Modeling: Newton-Euler Method

To simulate the Mako Surgical Robot's motion and response under different joint trajectories, a complete dynamic model is implemented in Simulink using the Newton-Euler formulation. This method enables recursive computation of joint torques based on current positions, velocities, and accelerations, along with the robot's mass properties and kinematics [1].

2.1. Symbolic Torque Model

The dynamic model is built from a symbolic formulation using MATLAB. The custom script `Snewton_euler_dynamics.m` computes the symbolic torque vector τ from the inputs $q, \dot{q}, \ddot{q}$ (joint positions, velocities, and accelerations).

The process begins by loading the robot's physical parameters—such as link masses, centers of mass, and inertia tensors—from a .mat file. The Denavit-Hartenberg (DH) parameters are symbolically defined through the `RIO()` function, and forward kinematics are computed symbolically to determine each link's position and orientation in space.

The Newton-Euler algorithm can be summed up into two main recursions [1]:

- Forward recursion: computes the angular and linear velocities and accelerations of each link in the manipulator, propagating from the base to the end-effector.
- Backward recursion: computes the forces and torques acting on each joint, propagating from the end-effector back to the base. The final joint torques τ are computed by projecting the resulting moments onto each joint's axis of motion.

Special care is taken to rotate vectors into appropriate coordinate frames, and to compute accelerations at the center of mass of each link. The final symbolic expression for τ is optionally optimized using 'optimizeExpressions' to simplify and speed up future evaluations.

The Newton-Euler method is also exported as a numeric function, `newton_euler_numeric.m`, for faster evaluation within the MATLAB environment, facilitating testing and validation without relying on symbolic computation.

2.3. Simulink Implementation

A custom Simulink model for the dynamics of the robot using the Newton-Euler formulation, named `Newton_Euler_Simul.slx`, was developed (Figure 3).

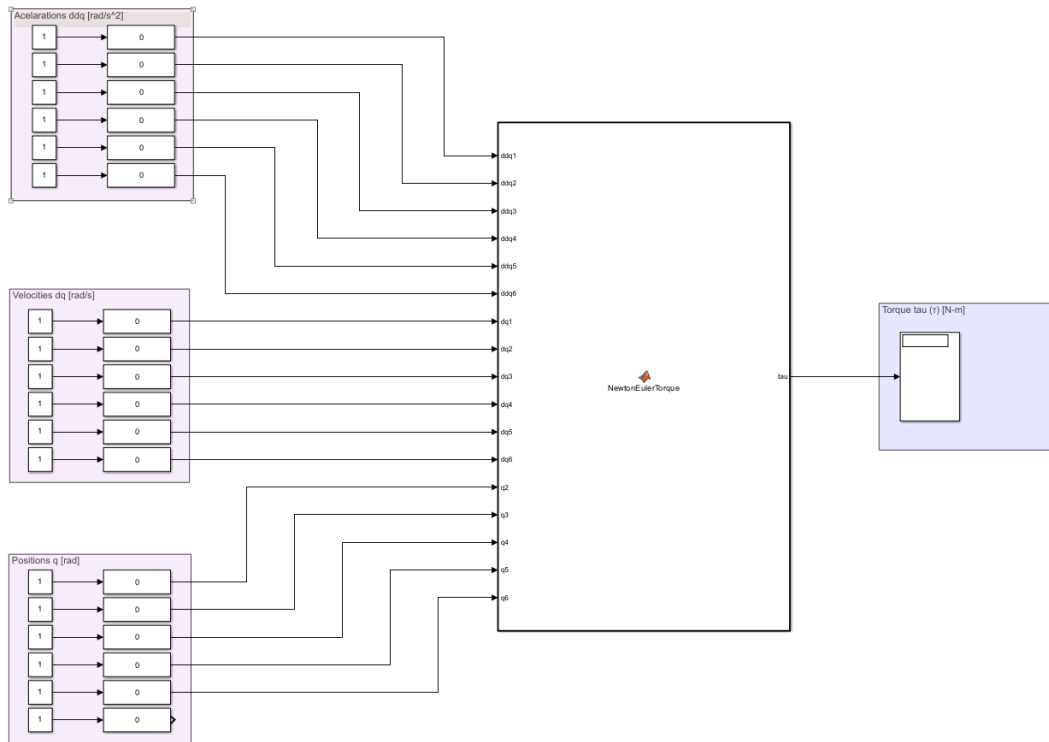


Figure 3: Robot Dynamics Newton-Euler formulation Simulink model

The model receives joints positions, velocities and accelerations as inputs (magenta areas in Figure 3) and computes the resulting torque vector (blue area in Figure 3) vector using Newton-Euler formulation.

The MATLAB function block `NewtonEulerTorque` (Figure 3) is generated by the script `newton_euler_Lib_example.m`. This script creates a MATLAB and Simulink code for the robot model, specifically its direct kinematics, and encapsulates it within a reusable Simulink library block. The block contains the simplified symbolic formula of the implementation of the `Snewton_euler_dynamics.m` function.

2.4. Newton-Euler Model Validation

In addition to the information provided in Appendix B of the project statement [2], to validate the Newton-Euler model, a simple test case was evaluated where all joint positions, velocities, and accelerations were set to zero (Figure 4). In this configuration, the only acting force should be gravity. With this configuration, it is expected that the computed joint torques reflect the gravitational loading of each link, with non-zero torque values appearing only in the joints that must counteract the downward force of gravity to maintain static equilibrium.

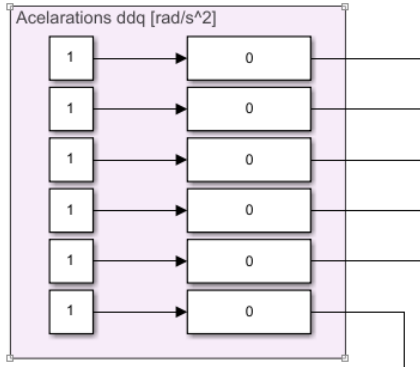


Figure 4: Simulink Newton-Euler model accelerations ddq_i set to 0 rad/s^2 . Gravity-only test scenario

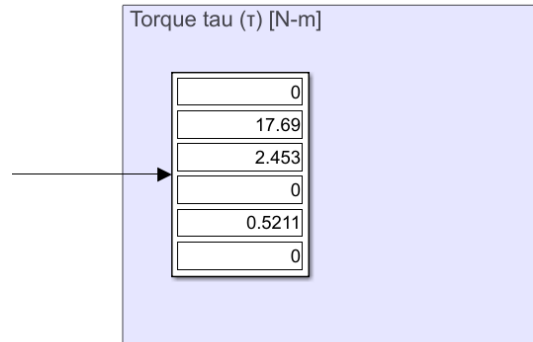


Figure 5: Output torque $[N.m]$ from the Simulink Newton-Euler model with all joint positions, velocities, and accelerations set to zero. Gravity-only test scenario

The results of the gravity-only test present in Figure 5 showed that joint 2 exhibited the largest absolute torque magnitude, as expected. This is due to the cumulative effect of all subsequent link masses acting through this joint, which must support most of the arm's weight in the vertical plane. Joints closer to the end-effector (e.g., joint 5) displayed significantly smaller absolute torque values, reflecting their role in orienting rather than supporting the major structural mass. Joints 1, 4, and 6 exhibit near-zero torque, which is consistent with the static nature of the test and their axes of rotation being aligned such that gravity induces no torque about them.

Although the exact torque values are harder to verify, the observed pattern is consistent with a static configuration under gravity and supports the model's ability to capture the expected load distribution.

3. Worst Case Inertia Configurations & Decentralized PID Controllers

This section addresses the computation of the worst-case inertial configuration for each joint of the robotic manipulator. The process involves evaluating the joint-space inertia matrix $M(q)$ derived from the symbolic Newton-Euler formulation, over a set of joint configurations. For each joint, the maximum value on the diagonal of $M(q)$ is identified, corresponding to the configuration that imposes the highest effective inertia on the actuator.

Based on these results, decentralized PID-type joint controllers are designed and implemented. The controllers are tuned to ensure adequate performance under the highest dynamic loads expected at each joint, providing a foundation for reliable motion control.

3.1. Joint Space Inertia Matrix Computation

The joint space inertia matrix $M(q)$ is computed symbolically using MATLAB (`S_inertia_matrix.m`), following a structured Newton-Euler formulation. The purpose of this matrix is to capture the mass and inertia effects of all the robot's links, as perceived from the base, in terms of the joint variables q .

The process begins by loading the robot's dynamic parameters, including mass, center of mass (CoM), and inertia tensors for each of the six links. The symbolic Denavit-Hartenberg (DH) table is used to compute the transformation matrices for each joint in terms of the symbolic joint variables.

For each link i , the transformation matrix from the base to the link frame is calculated iteratively. From this, the position of the CoM in the base frame is obtained. The linear and angular Jacobians J_v and J_w are then constructed for each link with respect to the base. These Jacobians are used to compute the contribution of each link to the total inertia matrix as:

$$M(q) = \sum_{i=1}^n (m_i J_{v_i}^T J_{v_i} + J_{w_i}^T R_i I_i R_i^T J_{w_i})$$

where:

- m_i is the mass of link i ,
- I_i is the inertia tensor of the link in its own frame,
- R_i is the rotation matrix from the link frame to the base frame.

The inertia tensors are rotated into the base frame using $R_i I_i R_i^T$ before being used in the computation. The resulting matrix is then simplified using MATLAB's symbolic toolbox with a limited number of steps ('Steps', 30) to reduce expression complexity and keep computation time reasonable.

This symbolic formulation provides an analytical expression for the inertia matrix, which can be evaluated numerically for any configuration, and is essential for model-based control and dynamic analysis.

To facilitate numerical evaluation, a section of the script `Worst_Inertias_Search.m` generates a numeric version of the matrix as the function `Numeric_Inertia_matrix.m`.

3.2. Worst Case Scenario Inertia Configuration

To design effective joint controllers, it is necessary to identify the configuration at which each joint experiences its maximum effective inertia. This is done by evaluating the diagonal elements $M_{ii}(q)$ of the inertia matrix over a range of joint positions. The configuration that maximizes each

M_{ii} represents the worst-case inertia for joint i , meaning it requires the highest torque to achieve a unit of acceleration. The maximum inertia configuration for each joint can be easily seen when the robot's arm is fully extended. This is because, in this configuration, the mass of each link is positioned farther from the base joints, increasing the moment of inertia due to the greater distance from the rotation axis. As a result, joints—especially those near the base—must overcome higher inertial resistance, making the dynamic effects more pronounced. The maximum inertias for each joint are the following:

$$\begin{aligned} I_{1 \max} &= 1.9891 \text{ kg/m}^2 \\ I_{2 \max} &= 1.9427 \text{ kg/m}^2 \\ I_{3 \max} &= 0.1584 \text{ kg/m}^2 \\ I_{4 \max} &= 0.4638 \text{ kg/m}^2 \\ I_{5 \max} &= 0.0791 \text{ kg/m}^2 \\ I_{6 \max} &= 0.0031 \text{ kg/m}^2 \end{aligned}$$

(Possible q configuration for the arm fully stretched: $[0 \quad 0 \quad 0 \quad \frac{\pi}{2} \quad 0 \quad \frac{\pi}{2}] \text{ rad}$)

The maximum joint inertia values reflect the expected mass distribution along the robot arm. Joints 1 and 2 exhibit the highest inertia, as they support the structure and mass of all subsequent links. A significant drop at joint 3 indicates a transition to lighter components, while the remaining joints show progressively smaller values.

3.3. Design and Implementation of Decentralized PID Joint Controllers

This approach treats each joint as a separate Single-Input Single-Output (SISO) system, ignoring cross-coupling effects for simplicity and tractability.

Joint Dynamics Modeling

Each joint was modeled as a second-order inertial system characterized by its moment of inertia I_j , derived from the symbolic inertia matrix computation (see Section 3.1). The simplified transfer function for joint j is given by:

$$G_j(s) = \frac{\Theta_j(s)}{\tau_j(s)} = \frac{1}{I_j s^2}$$

Where $\Theta_j(s)$ is the joint angle output and $\tau_j(s)$ is the input joint torque.

In accordance with the decentralized control approach, gravity and inter-joint coupling effects were neglected to model each joint independently as a second-order inertial system. This simplification allows for the design and tuning of individual PID controllers focused purely on acceleration and torque response without interference from other dynamics.

PID Controller Structure

Each joint controller is implemented as a standard PID feedback loop, where the error signal is the difference between desired and actual joint positions. The controller output generates the torque command τ_j to drive the joint. The PID control law is:

$$\tau_j(t) = K_{p,j} e_j(t) + K_{i,j} \int e_j(t) dt + K_{d,j} \frac{de_j(t)}{dt}$$

where $e_j(t)$ is the position error of joint j , and K_p , K_i , K_d are tuned proportional, integral, and derivative gains respectively.

Implementation and tuning in Simulink

This process was implemented in the Simulink model `PID_Joint_Controllers.slx` (Figure 6) for each one of the six joints. For each joint, the previously computed worst-case (maximum) the inertia value I_j was used as a parameter in the plant model block.

Note: The final 3 sections of the associated MATLAB script `Worst_Inertia_Search.m` must be executed to export these inertia values and the controller gain parameters for use in the simulation.

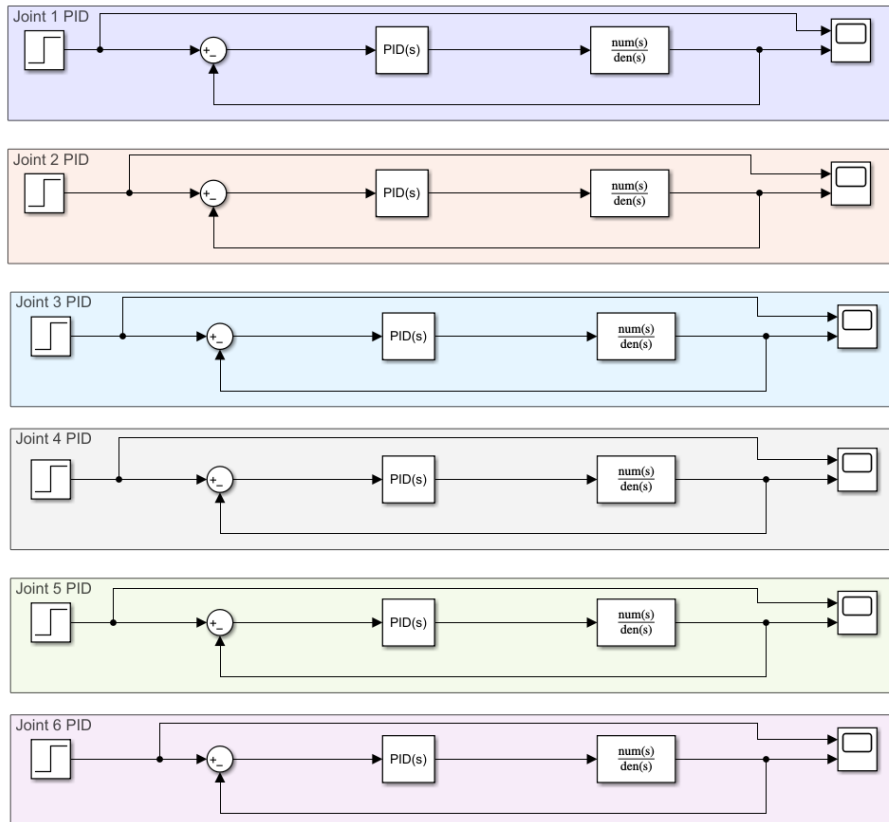


Figure 6: Simulink decentralized PID joint controllers

Controllers were tuned iteratively using unit step reference inputs to achieve desired response characteristics such as minimal overshoot, acceptable settling time, and reduced steady-state error.

The initial PID gains were chosen based on standard second-order system approximations. The gains were computed using target natural frequency ω_n and damping ratio ζ , yielding:

$$K_p = \omega_n^2 \cdot I_i \quad (4)$$

$$K_i = 0.1 * K_p \quad (5)$$

$$K_d = 2 \cdot \zeta \cdot \omega_n \cdot I_i \quad (6)$$

The natural frequency ω_n was set to 6 rad/s, a moderate response, generally suitable for robotic arms where joint movements must be responsive but not overly aggressive.

The damping ratio ζ was set to 0.7, as it represents a classic trade-off: it minimizes overshoot while still allowing for a relatively fast rise time and avoids underdamped oscillations.

The base filter coefficient N was set to 240 ($= 40\omega_n$). This value offers sufficient noise attenuation for the derivative term without significantly degrading the controller's responsiveness. Figure 7 shows Joint 1's response to the first gains approximation.

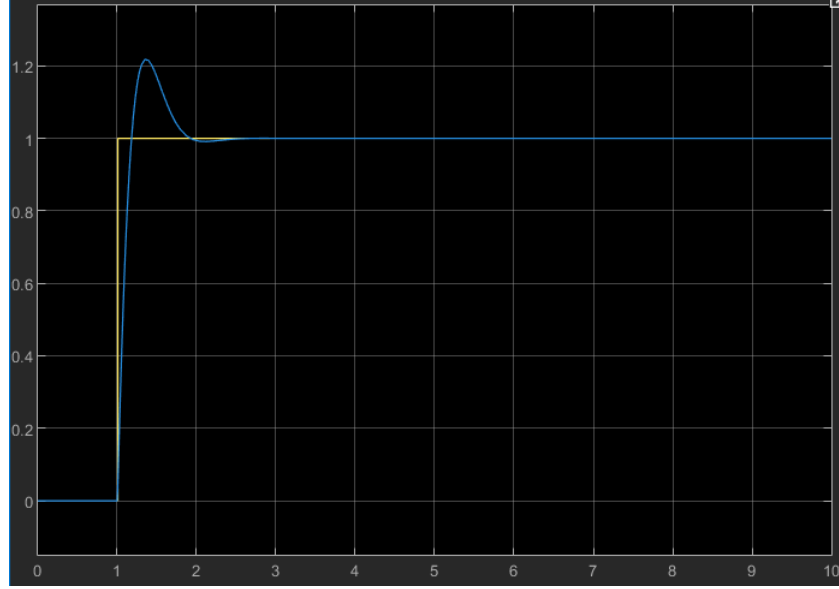


Figure 7: Joint 1 unit step response using initial PID gain approximation ($K_p = 71.6076$, $K_i = 7.1608$, $K_d = 16.7084$)

Curiously, the initial response was similar across all joints when using the gain formulas from Equations(4)–(6).

While the response was not poor, it still exhibited moderate overshoot and settling time that could be improved. This behavior indicated the need for further adjustment, particularly in tuning the proportional and derivative gains to refine transient performance. After several tuning iterations and evaluations, the best-performing gain formulas for all joints were found to be:

$$K_p = 3\omega_n^2 \cdot I_i \quad (7)$$

$$K_i = \frac{K_p}{T_i} \quad (8)$$

$$K_d = 9 \cdot \zeta \cdot \omega_n \cdot I_i \quad (9)$$

Based on the Equations (7)–(9), Table 2 presents the final PID values per joint.

Table 2: Final PID decentralized controller gains for each joint

Joint	K_p	K_i	K_d
1	214.8228	21.4823	75.1880
2	209.8116	20.9812	73.4341
3	17.1072	1.7107	5.9875
4	50.0904	5.0090	17.5316
5	8.5428	0.8543	2.9900
6	0.3348	0.0335	0.1172

Figure 8 shows the response of Joint 1 to a unit step reference using the final PID gains.

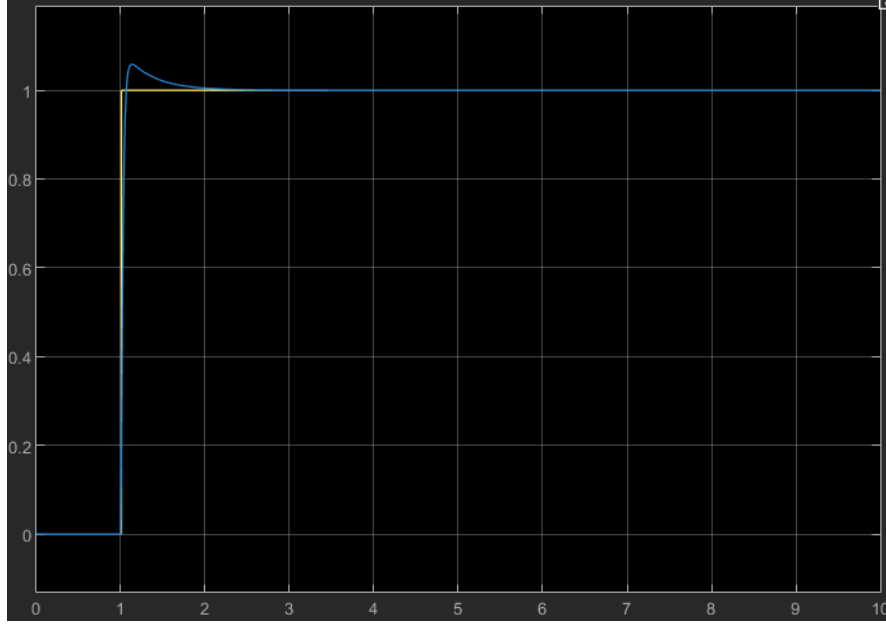


Figure 8: Joint 1 unit step response using final PID gain approximation ($K_p = 214.8228$, $K_i = 21.4823$, $K_d = 75.1880$)

Compared to the initial response, the tuned controller achieved minimal overshoot and settled within approximately 1.3 seconds, demonstrating a significant improvement in both stability and responsiveness for the individual joint. Similar behavior is verified across the other joints.

4. Centralized Inverse Dynamics Controller

In multi-joint manipulators such as the 6-DOF Mako RIO arm, motion in one joint affects the others due to dynamic coupling. Therefore, a centralized control approach is required to compensate for these interactions and ensure coordinated, accurate motion across all joints. [1]

4.1. Robot Dynamics Extraction

The full dynamic model of the robotic manipulator in the standard form is:

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) \quad (10)$$

where:

- $M(q)$ is the inertia matrix $M(q)$
- $C(q, \dot{q})$ is the Coriolis and centrifugal matrix
- $G(q)$ is the gravity vector

The inertia matrix $M(q)$ was previously computed previously (see Section 3.1.). Both the Coriolis matrix and the gravity vector were extracted using the `compute_dynamics.m` function. The Coriolis matrix $C(q, \dot{q})$ is computed from the Christoffel symbols, which are derived from the derivatives of $M(q)$. The gravity vector $G(q)$ is obtained by computing the

gradient of the potential energy with respect to each joint coordinate, considering the mass and center of mass of each link.

Both $C(q, \dot{q})$ and $G(q)$ were then exported as numerical functions (`Numeric_coriolisMatrix.m` and `Numeric_gravityVector.m`, respectively), allowing for faster evaluation and verification of the dynamic terms within the MATLAB environment.

4.2. Control Law Description and Implementation

Based on the derived dynamic model, the control law employed takes the form [1]:

$$\tau = M(q)(\ddot{q}_d + K_p(\dot{q}_d - \dot{q}) + K_d(q_d - q)) + C(q, \dot{q})\dot{q} + G(q)$$

Where:

- $\ddot{q}_d, \dot{q}_d, q_d$: desired position, velocity, acceleration
- K_p, K_d : diagonal proportional and derivative gain matrices

This is equivalent to cancelling the nonlinear dynamics and applying a linear PD control law to the resulting linearized system.

The symbolic formulation of the controller was implemented in the `computedTorqueControl_sym.m` function. This function computes the control torques τ as a function of joint states and desired trajectories. Additionally, the error between the desired and current positions and velocities are also computed in the function.

4.3. Direct Dynamics

Once the control torques τ have been computed by the inverse dynamics controller, the next step is to evaluate the system's response — that is, to compute the resulting joint accelerations $\ddot{q}$ using the full dynamic model of the manipulator. This step is commonly referred to as Direct or Forward Dynamics.

The direct dynamics equation is derived from equation (10) solving for $\ddot{q}$:

$$\ddot{q} = M(q)^{-1}[\tau - C(q, \dot{q})\dot{q} - G(q)]$$

This operation is implemented numerically in the `directDynamics_numeric.m` function.

Note: The Simulink simulation may take slightly longer to run the first time due to the numerical implementation of the `directDynamics_numeric.m` function.

4.4. Simulink Implementation

With the control law and dynamic model fully defined, the centralized inverse dynamics controller was implemented in Simulink, `Centralized_Inverse_Dynamics_Simul.slx` (Figure 9).

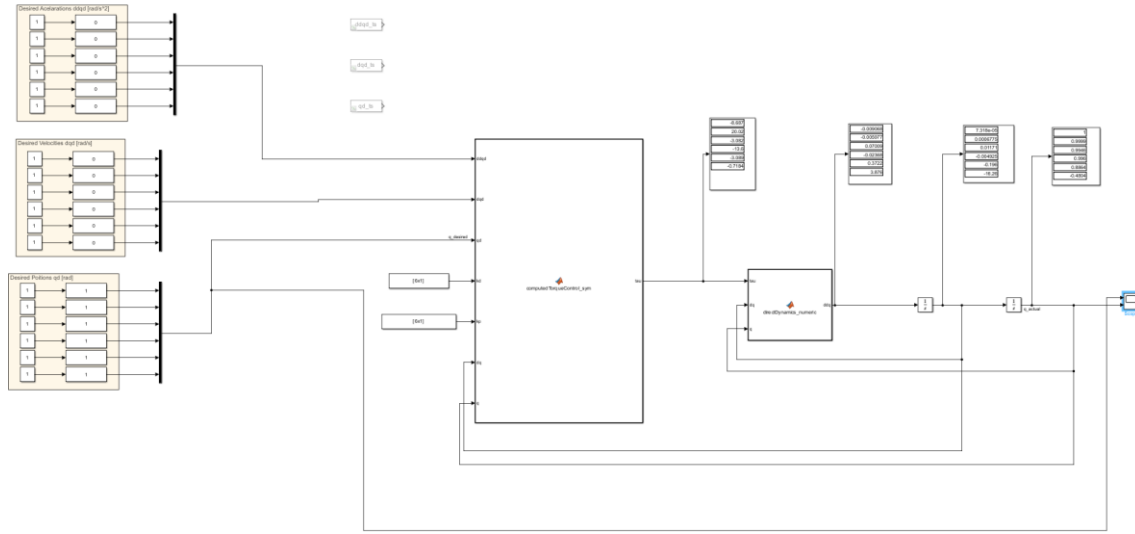


Figure 9: Implementation of the centralized inverse dynamics controller in Simulink

Both the computed control torque law and forward dynamics model were implemented using a MATLAB function blocks (see Figure 8). The computed torque law is first generated by the script `Centralized_Inverse_Dynamics_Lib example` where it is encapsulated within a reusable Simulink library block.

Due to computational limitations of symbolic evaluation during simulation, the forward dynamic model this function was manually inserted into a MATLAB Function block within Simulink. This allowed real-time evaluation of the system dynamics using numeric function calls instead of symbolic ones.

The control loop implemented in Figure 8 can be summarized as follows:

1. Desired trajectory input

Desired joint positions (q_d), velocities ($\dot{q}_d$), and accelerations ($\ddot{q}_d$) are generated or defined as reference inputs for the controller.

2. State Measurement/Estimation

The current joint positions (q) and velocities ($\dot{q}$) are obtained from the simulated model. In this case, they come from the two integrators, which emulate sensor feedback in a real system.

3. Error Computation

The control law calculates the position error $e = q_d - q$ and velocity error $\dot{e} = \dot{q}_d - \dot{q}$ (done inside the torque MATLAB function block).

4. Computed Torque Control

Using the inverse dynamics control law, the required joint torques (τ) are computed within the Torque MATLAB Function block. This block takes as inputs the desired joint trajectories (position, velocity, and acceleration) along with the PD gain values for each joint. It is important to note that the proportional and derivative gain values are each provided as separate 6×1 vectors, which are then assigned to the diagonals of their respective gains' matrices.

5. Forward Dynamics

The computed torques are applied to the system's dynamic model to calculate the resulting joint accelerations. This computation occurs inside the Direct Dynamics MATLAB Function block, which takes as inputs the previously computed torques, as well as the current joint positions and velocities (obtained from the integrators).

These current accelerations are integrated to update the velocities and positions.

6. Simulation Feedback

The updated joint states (current positions and velocities) are fed back into the controller and the dynamics block at each simulation time step, closing the control loop.

4.5. Controller Tuning and Validation

The gains of the feedback PD controller (embedded within the computed torque framework) were adjusted iteratively to achieve stable tracking with low overshoot and fast convergence, initially using step references. Subsequently, sinusoidal reference trajectories were also tested to further evaluate and refine the controller's performance.

The initial PD gains were chosen based on standard second-order system approximations. The proportional gain was set initially using Equation (4) and the derivative gain with Equation (6). The natural frequency ω_n was again set to 6 rad/s, and the damping ratio ζ was set to 0.7 for the same reasons discussed in Section 3.3..

Figure 10 shows the performance of the robot's joints set to the initial gain values.

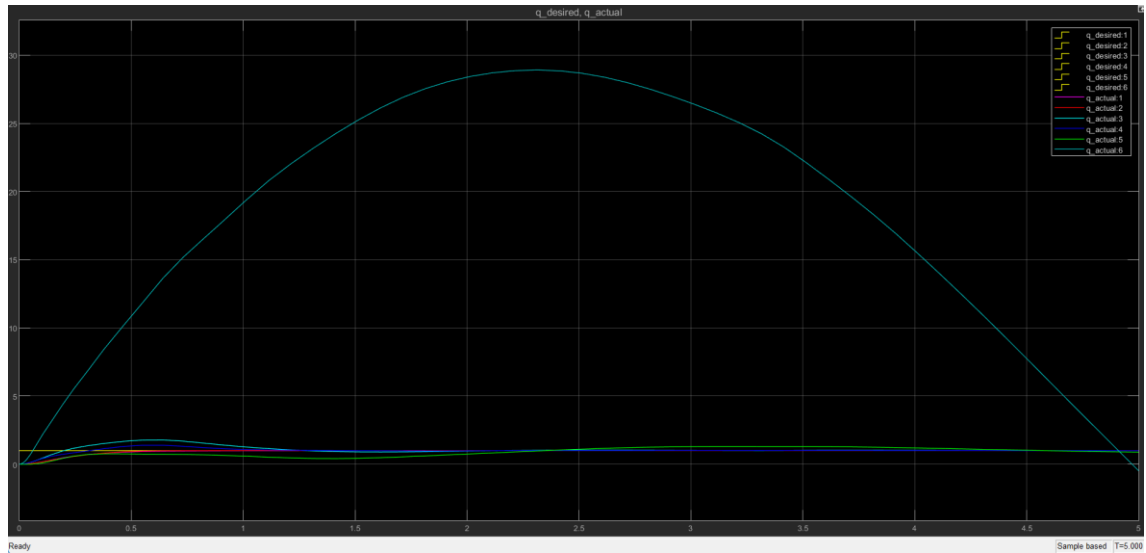


Figure 10: Joints unit step response using initial PD gain approximation

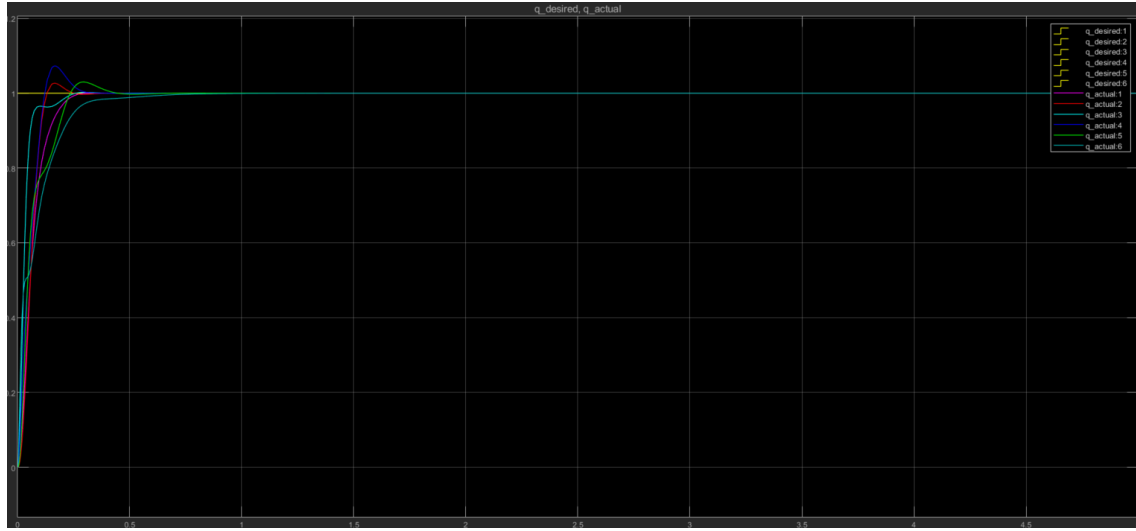
The initial joint responses clearly indicated poor performance, particularly for joint 6, which exhibited excessive overshoot and an unacceptably long settling time. In fact, joint 6's behavior suggested that it was not adequately controllable at all under the current PD structure. This observation highlighted the need for further tuning for all joints, especially for joint 6.

After multiple tuning iterations and evaluations, the best-performing gains for each joint were identified. Table 3 presents the final PD gains associated with each joint.

Tabela 3: Final PD centralized controller gains for each joint

Joint	K_p	K_d
1	900	60
2	750	40
3	750	30
4	450	30
5	375	30
6	375	70

Figure 11 shows the response of the joints to a unit step reference using the final PD gains.



The centralized controller proved more challenging to tune than the decentralized approach under a unit reference input, with gain values that deviated further from ideal.

Despite the tuning difficulties, the controller exhibited minimal overshoot and achieved settling within approximately 0.7 seconds for all joints. The primary challenge lay in adjusting the timing of when the reference was crossed. Overall, the controller produced reasonably good results.

5. End Effector Task-Space Trajectory Desing and Controller Performance Comparison

5.1. End Effector Task-Space Trajectory Design

To evaluate the closed-loop performance of both controllers — the decentralized PID and the centralized inverse dynamics (computed torque) — a smooth and realistic task-space motion was designed for the robot's end-effector. Specifically, the goal was to move the end-effector from an initial 3D position $p_A = [0.2, 0.1, 0.2]^T m$ to a final target $p_b = [0.6, 0.1, 0.2]^T m$ over a duration of 5 seconds, while maintaining constant orientation (identity rotation matrix and ensuring zero velocity at both the start and end of the motion).

To achieve the desired motion, the script `Trajectory_Design.m` was developed to generate a smooth joint-space trajectory corresponding to the task-space objective.

The CLICK inverse kinematics solver (developed in the first part of the project) was used to compute the initial and final joint-space configurations, q_A and p_B , associated with the desired end-effector poses.

It is worth noting that arbitrary values for q_A and q_b could have been defined directly. However, defining the trajectory in terms of end-effector position and orientation was considered more realistic and visually intuitive. This approach better reflects practical applications, despite the multiplicity of joint-space solutions for a given pose when using the CLICK inverse kinematics solver.

For each of the six joints, a cubic polynomial trajectory was constructed using the boundary conditions of zero initial and final joint velocities, and a fixed total movement duration of $T = 5$ seconds with a time step dt of 0.01 seconds. This approach guarantees smooth and continuous position, velocity, and acceleration profiles across all joints.

To facilitate simulation, the resulting joint trajectories $q(t)$, $\dot{q}(t)$ and $\ddot{q}(t)$ were precomputed and formatted as time-series objects. These time-stamped signals were then used as reference inputs in Simulink, enabling a direct and consistent performance comparison between the decentralized PID and the centralized inverse dynamics (computed torque) controllers.

Once the joint trajectories are generated, the `Trajectory_Design.m` script also provides visualizations of each joint's position, velocity, and acceleration over time, allowing for inspection and validation of the motion profile. Figure 12 displays the intended trajectory of joint 1.

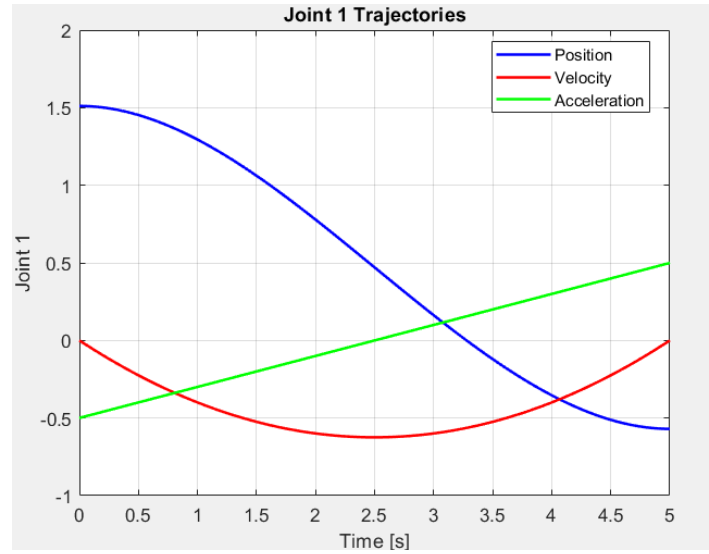


Figure 12: Position, velocity, and acceleration profiles of joint 1 over time, as generated by the trajectory design script.

The trajectory is smooth and accurately follows the desired specifications, particularly in terms of movement duration and boundary positions. The same can be verified for the other joints.

The trajectory designed and implemented in this section is an example of point-to-point motion using polynomial trajectory profiles. Although only two points (start and end) are considered here, the same approach—and the developed code—can be readily extended to handle motion through a sequence of via-points (motion through a sequence of points) [3].

Additionally, when precise verification of joint velocities and accelerations is needed (e.g., to stay within actuator limits), alternative trajectory profiles such as blended polynomials or trapezoidal velocity profiles may be more appropriate. In such cases, lower-order polynomials or piecewise-defined segments can offer better control over peak values and ensure smooth transitions.

5.2. Controller Performance Comparison

The desired trajectory was implemented using “From Workspace” blocks as inputs in Simulink.

Note: To avoid the need to regenerate the trajectory at each simulation run, the trajectory data (designed in Section 5.1.) was precomputed and saved into a .mat file named `precomputed_trajectory.mat`. This allows the model to load the joint position, velocity, and acceleration reference signals directly into the simulation environment without rerunning the generation script. (`load('precomputed_trajectory.mat')`).

To test each controller, the decentralized and centralized gain values listed in Tables 2 and 3, respectively, were used as input values for the simulation. Figures 13, 14 show the performance of the decentralized and centralized controllers for a initial reference of $[0; 0; 0; 0; 0; 0]$, respectively.

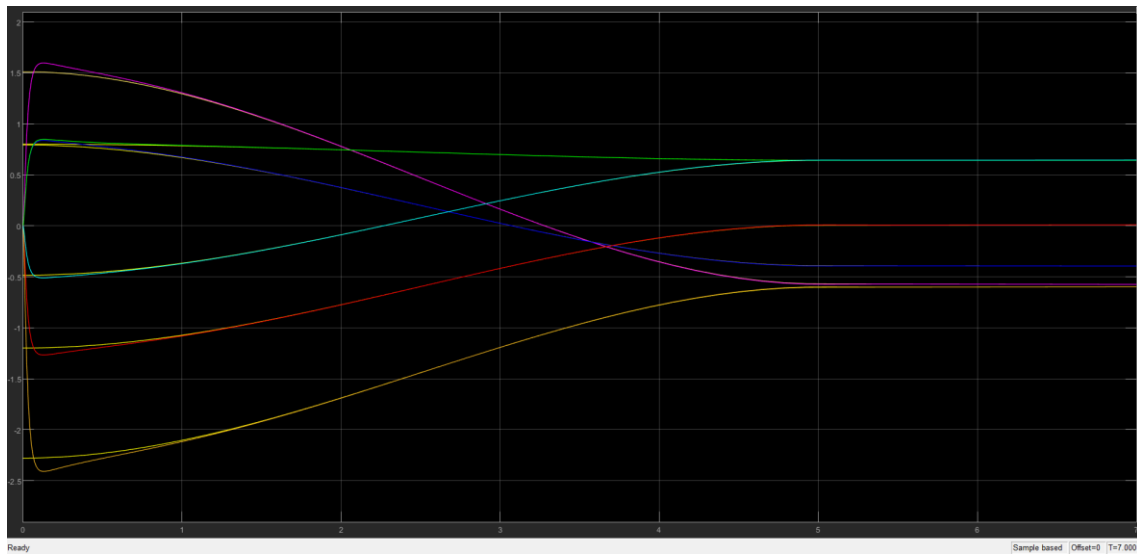


Figure 13: PID Decentralized controller performance on desired trajectory (initial reference $[0; 0; 0; 0; 0; 0]$)

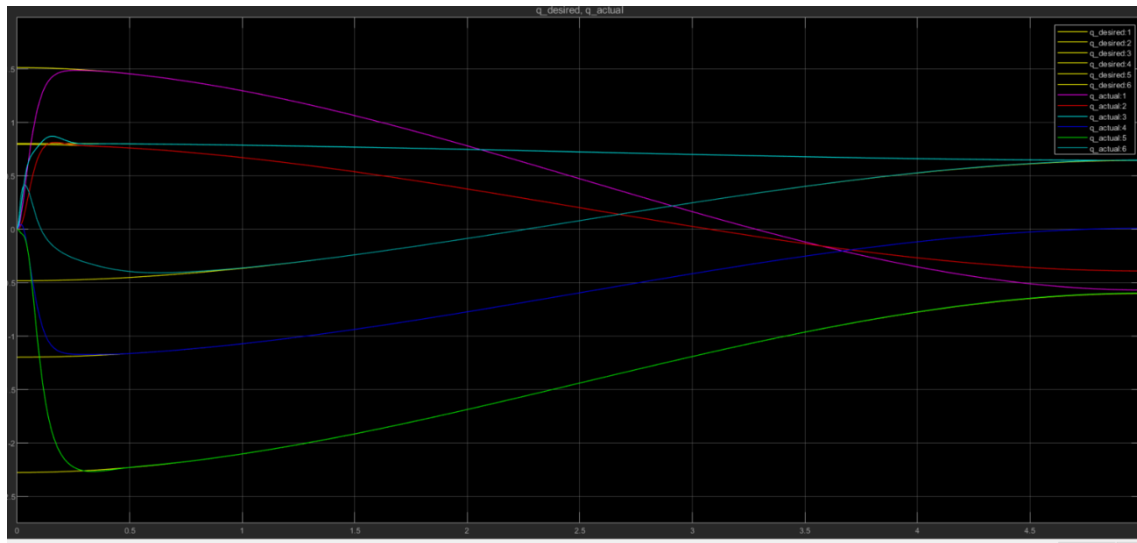


Figure 14: PD Centralized controller performance on desired trajectory (initial reference $[0; 0; 0; 0; 0; 0]$)

During the execution of the end-effector task, the joint-space decentralized PID control showed better tracking performance than the centralized Controller at a $[0; 0; 0; 0; 0; 0]$ initial reference. Specifically, the decentralized achieved lower tracking error and faster settling across all joints. Controller 4 exhibited more oscillation, particularly in joints 5 and 6, likely due to model sensitivity or inaccuracies. The superior performance of the decentralized controller might also be attributed to the fact that tuning is simpler when each joint is evaluated and controlled individually, making the system more robust to uncertainties. Overall, the decentralized controller delivered smoother joint trajectories and more reliable torque profiles for the task.

Figure 15 shows how the centralized controller would perform from an initial reference equal to the initial position q_A . It follows the trajectory very closely, demonstrating excellent tracking performance.

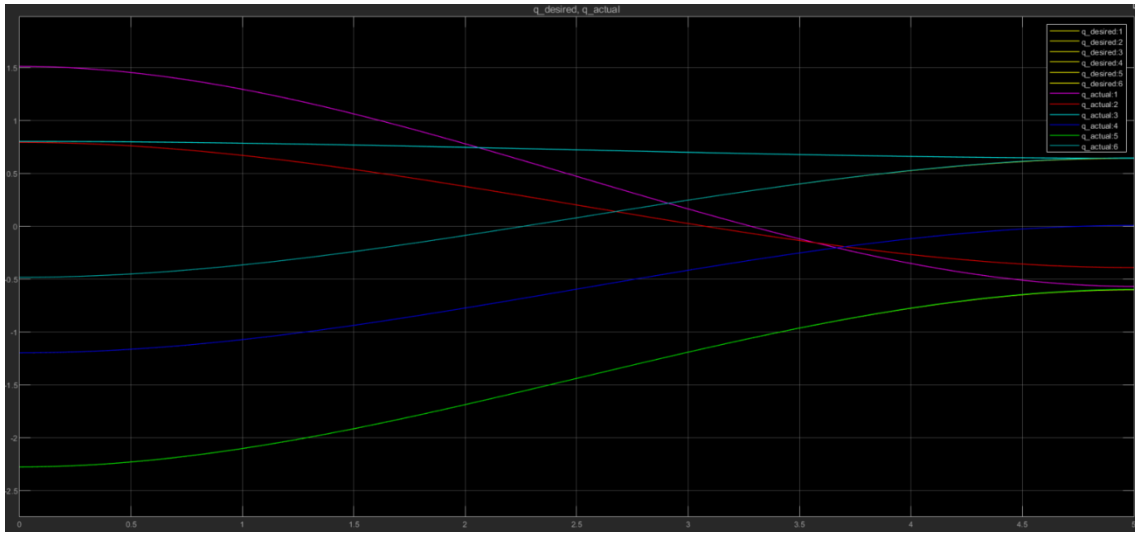


Figure 15: PD Centralized controller performance on desired trajectory (initial reference q_A)

Conclusion

This project successfully addressed the dynamic modeling and control of the Mako surgical robot using a structured approach grounded in the Newton-Euler formulation. Beginning with the estimation of physical parameters for each link—including mass, center of mass, and inertia tensor—the groundwork was laid for accurately simulating the robot’s motion under various operating conditions.

A symbolic dynamic model was developed in MATLAB and converted into a numeric function, which was then integrated into Simulink as a reusable block. This implementation enabled real-time computation of joint torques based on the current joint configuration, velocity, and acceleration. The model was validated through test cases designed to verify consistency between expected and computed torques.

To achieve effective control, both decentralized PID joint controllers and a centralized inverse dynamics controller were implemented. The decentralized approach offered simplicity and modularity, while the centralized controller utilized the full dynamic model to compute feedforward torques. A comparative analysis revealed the performance advantages of model-based control in trajectory tracking tasks.

Finally, a task-space trajectory was assigned to the robot’s end-effector, allowing for practical evaluation of controller performance. The simulation results confirmed that both control strategies could follow the desired trajectory, with the decentralized controller showing superior tracking accuracy possibly due to the simplicity of the simulation.

In summary, all simulations and control schemes were developed using original MATLAB and Simulink models, offering a complete and integrated framework for the dynamic analysis and control of the Mako surgical robot.

References

- [1] J. M. M. Martins and Professor Auxiliar, Robótica de Manipulação – Dynamics and Control. [PDF], Available on Fenix course page.
- [2] Martins, J., & Coelho, R. (2024). Trabalho laboratorial: Mako surgical robot, Robótica de Manipulação . Available on Fenix course page.
- [3] J. M. M. Martins, Kinematics and Trajectory Planning. [PDF], Available on Fenix course page.